```
    // assign ob1 to ob2
    ob2 = ob1;

    cout << "Here is ob1's a and b: ";
    cout << ob1.get_a() << " " << ob1.get_b() << "\n";

    cout << "Here is ob2's a and b: ";
    cout << ob2.get_a() << " " << ob2.get_b() << "\n";

    /*
       As you can probably guess, the output is the same for
       each object.
    */

    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 4

1. When one object is assigned to another of the same type, the current values of all data members of the object on the right are assigned to the corresponding data members on the left.

2. Trouble can occur when one object is assigned to another if that assignment overwrites important data already existing in the target object. For example, a pointer to dynamic memory or to an open file can be overwritten and, therefore, lost.

3. When an object is passed to a function, a copy is made. However, the copy's destructor is called when the object is destroyed by the termination of the function.

4. The violation of the separation between an argument and its copy when passed to a parameter can be caused by several destructor, that memory will also be lost to the argument. In general, if the destructor function destroys anything that the original argument requires, damage to the argument will occur.

5.
```
#include <iostream>
using namespace std;

class summation
{
    int num ;
    long sum ; // summation of num
public:
    void set_sum ( int n);
    void show_sum ()
    {
        cout << num << " summed is " << sum << "\n";
    }
};

void summation :: set_sum (int n)
{
```

```
    int i;
    num = n;
    sum = 0;
    for (i =1; i <=n; i ++)
        sum += i;
}

summation make_sum()
{
    int i;
    summation temp;

    cout << "Enter number: ";
    cin >> i;

    temp.set_sum(i);

    return temp;
}

int main()
{
    summation s;

    s = make_sum();

    s.show_sum();

    return 0;
}
```

6. For some compilers, in-line functions cannot contain loops.

7.
```
#include <iostream>
using namespace std;

class myclass
{
    int num;
public:
    myclass ( int x)
    {
        num = x;
    }
    friend int isneg(myclass ob);
};

int isneg(myclass ob)
{
    return (ob.num <0) ? 1 : 0;
}
```

```
int main()
{
    myclass a(-1), b(2);

    cout << isneg(a) << ' ' << isneg(b);
    cout << '\n';

    return 0;
}
```

8. Yes, a friend function can be friends with more than one class.

4.1 EXERCISES

1.
```
#include <iostream>
using namespace std;

class letters
{
    char ch;
public:
    letters(char c) { ch = c; }
    char get_ch() { return ch; }
};

int main()
{
    letters ob[10] = { 'a', 'b', 'c', 'd', 'e', 'f',
                       'g', 'h', 'i', 'j' };
    int i;

    for(i=0; i<10; i++)
        cout << ob[i].get_ch() << ' ';

    cout << "\n";

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

class squares
{
    int num , sqr ;
public:
    squares ( int a, int b) { num = a; sqr = b; }
    void show () { cout << num << ' ' << sqr << "\n"; }
};

int main()
```

```
{
    squares ob[10] = {
        squares(1, 1),
        squares(2, 4),
        squares(3, 9),
        squares(4, 16),
        squares(5, 25),
        squares(6, 36),
        squares(7, 49),
        squares(8, 64),
        squares(9, 81),
        squares(10, 100),
        };
    int i;

    for(i=0; i<10; i++)
        ob[i].show();

    return 0;
}
```

3.
```
#include <iostream>
using namespace std;

class letters
{
    char ch;
public:
    letters(char c) { ch = c; }
    char get_ch() { return ch; }
};

int main()
{
    letters ob[10] = {
        letters('a'),
        letters('b'),
        letters('c'),
        letters('d'),
        letters('e'),
        letters('f'),
        letters('g'),
        letters('h'),
        letters('i'),
        letters('j'),
    };
    int i;

    for(i=0; i<10; i++)
        cout << ob[i].get_ch() << ' ';
```

```
    cout << "\n";

    return 0;
}
```

4.2 EXERCISES

1.
```
// Display in reverse order.
# include <iostream>
using namespace std;

class samp
{
    int a, b;
public :
    samp ( int n, int m) { a = n; b = m; }
    int get_a () { return a; }
    int get_b () { return b; }
};

int main ()
{
    samp ob [4] = {
        samp(1 , 2),
        samp(3 , 4),
        samp(5 , 6),
        samp(7 , 8)
    };
    int i;

    samp *p;

    p = &ob[3]; // get address of last element

    for(i=0; i<4; i++)
    {
        cout << p->get_a() << ' ';
        cout << p->get_b() << '\n';
        p--; // advance to previous object
    }

    cout << "\n";
    return 0;
}
```

2.
```
/*
   Create a two-dimensional array of objects.
   Access via a pointer.
*/
# include <iostream>
```

```
using namespace std ;

class samp
{
    int a;
public :
    samp ( int n) { a = n; }
    int get_a () { return a; }
};

int main ()
{
    samp ob [4][2] = { 1, 2,
                       3, 4,
                       5, 6,
                       7, 8,
                     };
    int i;

    samp *p;

    p = (samp *) ob;
    for(i=0; i<4; i++)
    {
        cout << p->get_a() << ' ';
        p++;
        cout << p->get_a() << "\n";
        p++;
    }

    cout << "\n";
    return 0;
}
```

4.3 EXERCISE

1.
```
// Use this pointer.
#include <iostream>
using namespace std;

class myclass
{
    int a, b;
public:
    myclass(int n, int m) { this->a = n; this->b = m; }
    int add() { return this->a + this ->b; }
    void show();
};

void myclass::show()
{
```

```
    int t;

    t = this->add(); // call member function
    cout << t << "\n";
}

int main()
{
    myclass ob(10, 14);

    ob.show();

    return 0;
}
```

4.4 EXERCISES

1.
```
#include <iostream>
using namespace std;

int main()
{
    float *f;
    long *l;
    char *c;

    f = new float;
    l = new long;
    c = new char;

    if(!f || !l || !c)
    {
        cout << "Allocation error.";
        return 1;
    }

    *f = 10.102;
    *l = 100000;
    *c = 'A';

    cout << *f << ' ' << *l << ' ' << *c;
    cout << '\n';

    delete f;
    delete l;
    delete c;

    return 0;
}
```

2.
```
#include <iostream>
#include <cstring>
using namespace std;

class phone
{
    char name[40];
    char number[14];
public:
    void store(char *n, char *num);
    void show();
};

void phone::store(char *n, char *num)
{
    strcpy(name, n);
    strcpy(number, num);
}

void phone::show()
{
    cout << name << ": " << number;
    cout << "\n";
}

int main()
{
    phone *p;

    p = new phone;

    if(!p)
    {
        cout << "Allocation error.";
        return 1;
    }

    p->store("Isaac Newton", "111 555-2323");

    p->show();

    delete p;

    return 0;
}
```

3. On failure, **new** will either return a null pointer or generate an exception. You must check your compiler's documentation to determine which approach is used. In Standard C++, **new** generates an exception by default.

4.5 EXERCISES

1.
```
char *p;

p = new char [100];
// ...
strcpy(p, "This is a test");
```

2.
```
#include <iostream>
using namespace std;

int main()
{
    double *p;

    p = new double (-123.0987);

    cout << *p << '\n';

    return 0;
}
```

4.6 EXERCISES

1.
```
#include <iostream>
using namespace std;

void rneg(int &i); // reference version
void pneg(int *i); // pointer version

int main()
{
    int i = 10;
    int j = 20;

    rneg(i);
    pneg(&j);

    cout << i << ' ' << j << '\n';

    return 0;
}

// using a reference parameter
void rneg(int &i)
{
    i = -i;
}

// using a pointer parameter
void pneg(int *i)
```

```
{
     *i = -*i;
}
```

2. When **triple()** is called, the address of **d** is explicitly obtained with the **&** operator. This is neither necessary nor legal. When a reference parameter is used, the argument is not preceded by the **&**.

3. The address of a reference parameter is automatically passed to the function. You need not obtain the address manually. Passing by reference is often faster than passing by value. No copy of argument is generated. Therefor, there is no chance of a side effect occurring because the copy's destructor is called.

4.7 EXERCISE

1. In the original program, the object is passed to **show()** by value. Thus, a copy is made. When **show()** returns, the copy is destroyed and its destructor is called. This causes **p** to be released, but the memory pointed to it is still needed by the arguments to **show()**. Here is a corrected version that uses a reference parameter to prevent a copy from being made when the function is called:

```
// This program is now fixed.
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
     char *p;
public:
     strtype(char *s);
     ~strtype() { delete [] p; }
     char *get() { return p; }
};

strtype::strtype(char *s)
{
     int l;
     l = strlen(s)+1;

     p = new char [l];
     if(!p)
     {
          cout << "Allocation error\n";
          exit(1);
     }

     strcpy(p, s);
}

// Fix by using a reference parameter.
```

```
void show(strtype &x)
{
     char *s;

     s = x.get();
     cout << s << "\n";
}

int main()
{
     strtype a("Hello"), b("There");

     show(a);
     show(b);

     return 0;
}
```

4.8 EXERCISES

1.
```
// A simple bounded two-dimensional array example.
#include <iostream>
#include <cstdlib>
using namespace std;

class array
{
     int isize, jsize;
     int *p;
public:
     array(int i, int j);
     int &put(int i, int j);
     int get(int i, int j);
};

array::array(int i, int j)
{
     p = new int [i*j];
     if(!p)
     {
          cout << "Allocation error\n";
          exit(1);
     }
     isize = i;
     jsize = j;
}

// Put something into the array
int &array::put(int i, int j)
{
     if(i<0 || i>=isize || j<0 || j>=jsize)
```

```
    {
        cout << "Bounds error!!!\n";
        exit(1);
    }
    return p[i*jsize + j]; // return reference to p[i]
}

// Get something from the array
int array::get(int i, int j)
{
    if(i<0 || i>=isize || j<0 || j>=jsize)
    {
        cout << "Bounds error!!!\n";
        exit(1);
    }
    return p[i*jsize + j]; // return character
}

int main()
{
    array a(2, 3);
    int i, j;

    for(i=0; i<2; i++)
        for(j=0; j<3; j++)
            a.put(i, j) = i*j;

    for(i=0; i<2; i++)
        for(j=0; j<3; j++)
            cout << a.get(i, j) << " ";

    // generate out of bounds
    a.put(10, 10);

    return 0;
}
```

2. No. A reference returned by a function cannot be assigned to a pointer.

MASTERY SKILLS CHECK: Chapter 4

1.
```
#include <iostream>
using namespace std;

class a_type
{
    double a, b;
public:
    a_type(double x, double y)
    {
        a = x;
```

```
        b = y;
    }
    void show() { cout << a << ' ' << b << '\n'; }
};

int main()
{
    a_type ob[2][5] = {
        a_type(1, 1), a_type(2, 2),
        a_type(3, 3), a_type(4, 4),
        a_type(5, 5), a_type(6, 6),
        a_type(7, 7), a_type(8, 8),
        a_type(9, 9), a_type(10, 10),
    };

    int i, j;

    for(i=0; i<2; i++)
        for(j=0; j<5; j++)
        ob[i][j].show();

    cout << '\n';

    return 0;
}
```

```
#include <iostream>
using namespace std;

class a_type
{
    double a, b;
public:
    a_type(double x, double y)
    {
        a = x;
        b = y;
    }
    void show() { cout << a << ' ' << b << '\n'; }
};

int main()
{
    a_type ob[2][5] = {
        a_type(1, 1), a_type(2, 2),
        a_type(3, 3), a_type(4, 4),
        a_type(5, 5), a_type(6, 6),
        a_type(7, 7), a_type(8, 8),
        a_type(9, 9), a_type(10, 10),
    };

    a_type *p;
```

```
    p = (a_type *) ob;

    int i, j;

    for(i=0; i<2; i++)
    {
        for(j=0; j<5; j++)
        {
            p->show();
            p++;
        }
    }

    cout << '\n';

    return 0;
}
```

2. The **this** pointer is a pointer that is automatically passed to a member function and that points to the object that generated the call.

3. The general forms of **new** and **delete** are:

```
p_var = new type;
delete p_var;
```

   When using **new**, you don't need to use a type cast. The size of the object is automatically determined; you don't need to use **sizeof**0 Also, you don't need to include **<cstdlib>** in you program.

4. A reference is essentially an implicit pointer constant that is effectively a different name for another variable or argument. One advantage of using a reference parameter is that no copy of the argument is made.

5.
```
#include <iostream>
using namespace std;

void recip(double &d);

int main()
{
    double x = 100.0;

    cout << "x is " << x << '\n';

    recip(x);

    cout << "Reciprocal is " << x << '\n';

    return 0;
}

void recip(double &d)
```

```
{
    d = 1/d;
}
```

CUMULATIVE SKILLS CHECK: Chapter 4

1. To access a member of an object by using a pointer, use the arrow (-> operator.

2. ```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *ptr);
    ~strtype();
    void show();
};

strtype::strtype(char *ptr)
{
    len = strlen(ptr);

    p = new char [len+1];
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }

    strcpy(p, ptr);
}

strtype::~strtype()
{
    cout << "Freeing p\n";
    delete [] p;
}

void strtype::show()
{
    cout << p << " - length: " << len;
    cout << '\n';
}

int main()
{
```
```

```
    strtype s1("This is a test."), s2("I like C++");

    s1.show();
    s2.show();

    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 5

1. A reference is a special type of pointer that is automatically dereferenced and that can be used interchangeably with the object it is pointing to. There are three types of references: parameter references, independent reference, and references that are returned by functions.

2. ```
   #include <iostream>
   using namespace std;

   int main()
   {
       float *f;
       int *i;

       f = new float;
       i = new int;

       if(!f || !i)
       {
           cout << "Allocation error\n";
           return 1;
       }

       *f = 10.101;
       *i = 100;

       cout << *f << ' ' << *i << '\n';

       delete f;
       delete i;

       return 0;
   }
   ```

3. The general form of **new** that includes an initializer is shown here:

   ```
   p_var = new type (initializer);
   ```

   For example, this allocates an integer and gives it the value 10:

   ```
   int *p;

   p = new int (10);
   ```
   ```